SentinelNode — Part 3: Advanced Features & Industry Research

Learning notes from the live build session

This document covers: the health check API gateway pattern, Shannon entropy detection, the capture group bug fix, SSE streaming architecture, the filesystem watcher and its WSL2 limitations, the ignore list feature, and a thorough industry research comparison of our workflow against vibe coding norms as of March 2026. Read alongside Parts 1 and 2.

Contents

1 Health Check — API Gateway Pattern **3**
- 1.1 Why internal services never talk directly to the browser . . . 3
- 1.2 Implementation . . . 3

2 SSE Streaming Architecture **3**
- 2.1 Why EventSource cannot use auth middleware . . . 3
- 2.2 Buffer parsing for SSE chunks . . . 4
- 2.3 The scan thread and tiny_http blocking . . . 4

3 Shannon Entropy Detection **4**
- 3.1 Theory . . . 4
- 3.2 The capture group bug . . . 4
- 3.3 Stopwords . . . 5
- 3.4 Why [0-9a-f]{64} was removed . . . 5

4 The Rule Architecture **5**
- 4.1 Structure of a Rule . . . 5
- 4.2 Named rules implemented . . . 6
- 4.3 Three layers of false positive defence . . . 6

5 Filesystem Watcher and WSL2 Limitations **6**
- 5.1 The notify crate . . . 6
- 5.2 Why the watcher is disabled in v0.2.0 . . . 6
- 5.3 Why full drive scanning crashes Docker . . . 6

6 Configurable Ignore List **6**
- 6.1 Architecture . . . 6
- 6.2 UI design rationale . . . 7

7 Industry Research: Vibe Coding vs Our Workflow **7**
- 7.1 What most vibe coders get wrong . . . 7
- 7.2 What we did instead . . . 7
- 7.3 The security gap . . . 8
- 7.4 What the research says separates good AI-assisted development . . . 8
- 7.5 The honest gap . . . 8
- 7.6 Bottom line . . . 8

8 v0.2.0 Feature Summary **9**

Health Check — API Gateway Pattern

Why internal services never talk directly to the browser

The health check for `rust-core` is proxied through Node rather than exposed directly on port 8080. This is the API Gateway pattern and it is the architectural rule that governs every service boundary in the project: **internal services never talk directly to the browser. Only the API gateway does.**

Exposing port 8080 to the host would mean `rust-core` is reachable by anyone on the local network with no authentication, no rate limiting, and no error handling. As new internal services are added in future versions, each one would grow the attack surface. By contrast, every request going through Node gets auth middleware, logging, and consistent error handling automatically.

It also means the frontend never changes if the internal implementation changes. If `rust-core` is replaced or a second scanning service is added, only the Node routes change — the browser always talks to `localhost:3000`.

Implementation

The data flow is:

```
Browser -> node-api:3000/api/scanner/health -> rust-core:8080/health
```

`rust-core` answers `GET /health` with `{"status":"ok"}` instantly. Node proxies this with a 3-second `AbortController` timeout. If `rust-core` is unreachable, Node returns 503.

`CoreStatusDot` in `Dashboard.tsx` polls this every 30 seconds using TanStack Query's `refetchInterval`. Three visual states: gray pulsing dot while checking, green dot when online, red dot when offline. The scan button is disabled when offline with a clear message explaining why.

> The `coreHealth` query key is shared between `CoreStatusDot` in `Dashboard.tsx` and `ScanResults.tsx`. TanStack Query deduplicates requests — both components share one network call rather than each making their own.

SSE Streaming Architecture

Why EventSource cannot use auth middleware

Server-Sent Events use the browser's built-in `EventSource` API. Unlike `fetch`, `EventSource` does not support a `credentials` option or custom headers. This means the standard Hono `authMiddleware` cannot intercept SSE connections — the middleware chain is bypassed entirely.

The fix is to verify the JWT cookie manually inside the stream handler itself, using the shared `verifyToken` utility exported from `auth.ts`. The cookie still arrives automatically (browsers always send cookies on same-origin requests) — we just verify it ourselves rather than relying on the middleware.

This is the correct architectural answer because verification logic stays in one place (`auth.ts`) and the stream handler borrows it. The alternative — duplicating JWT verification code in the handler — would be a copy-paste violation.

Buffer parsing for SSE chunks

TCP does not guarantee that SSE events arrive one at a time. A single `read()` call from Node's stream reader may contain multiple events, or a single event may be split across two reads. The correct approach is to accumulate chunks in a string buffer and split on the `\n\n` delimiter that separates SSE events:

```
buffer += decoder.decode(value, { stream: true });
const lines = buffer.split("\n\n");
buffer = lines.pop() ?? ""; // incomplete event stays in buffer
for (const line of lines) {
  // process complete events
}
```

The scan thread and tiny_http blocking

`tiny_http` is a single-threaded HTTP server. If the scan runs on the main HTTP thread, no other requests (including health checks) can be processed until the scan finishes. The fix is to spawn the scan on a separate thread with a 64 MB stack and communicate back through a `crossbeam_channel`:

```
std::thread::Builder::new()
    .stack_size(64 * 1024 * 1024)
    .spawn(move || {
        // scan runs here
        let _ = progress_tx.send(ProgressEvent::Complete { result });
    }).expect("Failed to spawn scan thread");

// main HTTP thread just reads from the channel
for event in progress_rx { ... }
```

The main thread blocks on the channel read, forwarding SSE events to Node as they arrive. Health check requests are handled by a separate OS thread because `tiny_http` uses one thread per connection.

Shannon Entropy Detection

Theory

Shannon entropy measures how random a string is. It is computed as:

$$H = -\sum_i p_i \log_2 p_i$$

where p_i is the frequency of each unique character divided by the string length. The maximum entropy for a hex string (16 possible characters) is $\log_2(16) = 4.0$. For base64 (64 possible characters) it is $\log_2(64) = 6.0$.

In practice: real secrets score 3.5–4.8. Placeholders like `changeme` score around 2.75. The string `JWT_SECRET` scores 2.92. A 64-char hex string generated by `openssl rand -hex 32` scores approximately 3.7.

The capture group bug

The `env-secret` rule uses the pattern:

```
^(JWT_SECRET|JWT_REFRESH_SECRET|...)=([A-Za-z0-9+/=_-]{20,})
```

This has two capture groups. Group 1 is the key name (`JWT_SECRET`). Group 2 is the secret value (the actual hex string).

The original code passed `m.get(1)` to `apply_rule` for entropy scoring. `JWT_SECRET` has entropy 2.92 — below the 3.5 threshold — so every `.env` secret was silently discarded.

The fix is to always score the *last* capture group, which contains the secret value rather than the identifier:

```
let last_group = m.len() - 1;
let matched = m.get(last_group)
    .or_else(|| m.get(0))
    .map(|m| m.as_str())
    .unwrap_or("");
```

The rule: when writing detection rules with multiple capture groups, always score entropy on the value group, not the identifier group. The identifier is context for the regex to anchor the match — it is never the secret itself.

Stopwords

Stopwords are the most effective false positive filter. If the matched value contains any of the words in the stopword list, the finding is discarded immediately without computing entropy:

```
"example", "test", "dummy", "placeholder", "changeme",
"your_", "yourkey", "xxx", "yyy", "zzz", "sample",
"fake", "mock", "todo", "fixme", "replace", "insert",
"change_me", "change_this", "random_64", "hex_string"
```

This eliminates documentation and template credentials before entropy scoring even runs. Per-rule stopwords handle service-specific cases (e.g. the AWS example key `AKIAIOSFODNN7EXAMPLE`).

Why [0-9a-f]{64} was removed

The pattern `[0-9a-f]{64}` matches any 64-character hex string. This includes Next.js cache filenames, git object hashes, content digests, and SHA-256 checksums. It produced hundreds of false positives on a real Desktop scan and was removed entirely. No professional scanner uses this pattern for secret detection.

The Rule Architecture

Structure of a Rule

Each rule in `scanner.rs` combines four things that together achieve near-zero false positives:

Field	Purpose
`pattern`	Regex anchored with `\b` word boundaries to prevent partial matches
`entropy_min`	Minimum Shannon entropy of the matched value. Lower for highly specific patterns (AWS: 3.0), higher for generic patterns (API key: 3.8)
`severity`	Critical / High / Medium assigned per service type
`stopwords`	Service-specific false positive overrides

Named rules implemented

AWS access key, AWS secret key, GitHub PAT, GitHub OAuth, GitHub Actions, Stripe secret, Stripe publishable, Google API key, Slack token, NPM token, SendGrid API key, Twilio SID, private key header, environment variable secret, and generic API key in code (14 rules total).

Three layers of false positive defence

1. **Skip lists** — directories (`.venv`, `node_modules`, `.next`, `patterns`, `fixtures`), file extensions (`.exe`, `.lock`, `.map`), and example/template files (`.env.example`, `.sample`) are never opened.

2. **Line filter** — lines containing `(?i)`, `\s*`, or `[A-Z0-9]{16}` are skipped. These are lines that contain regex patterns as strings, not actual secrets.

3. **Rule application** — stopword check first (fast string scan), entropy check second (mathematical), finding emitted only if both pass.

Filesystem Watcher and WSL2 Limitations

The notify crate

The `notify` crate wraps OS-specific filesystem watching APIs: `inotify` on Linux, `FSEvents` on macOS, and `ReadDirectoryChangesW` on Windows. Inside Docker on WSL2, it uses `inotify`.

Linux has a system limit on the number of `inotify` watchers. The default is 8,192 watches per user. Watching the entire C drive through WSL2 immediately exhausts this limit, causing `notify` to panic and kill the `rust-core` process.

Why the watcher is disabled in v0.2.0

The watcher is compiled and present in `watcher.rs` but commented out in `main.rs`. It will be re-enabled in v0.3.0 with OS-specific handling: on Linux/WSL2, raise the `fs.inotify.max_user_watches` limit before starting the watcher, or watch only specific subdirectories rather than the entire `SCAN_PATH`.

Why full drive scanning crashes Docker

`WalkDir` collects the entire directory tree into a `Vec` before Rayon processes it. On a full C drive with hundreds of thousands of files, this exhausts the container's memory limit. Docker sends SIGKILL — which bypasses Rust's panic hook entirely, so no error message appears in the logs.

The v0.3.0 fix is streaming `WalkDir` processing: process files in batches as they are discovered rather than collecting them all first.

Configurable Ignore List

Architecture

The ignore list is stored in SQLite in the `ignore_list` table with a `UNIQUE(user_id, pattern)` constraint. Three types are supported: `directory`, `extension`, and `file`, matching the three skip mechanisms already in `scanner.rs`.

The flow on each scan:

```
1. Node fetches ignore list from SQLite for current user
2. Builds IgnoreList { directories, extensions, files }
3. Sends it to rust-core as part of the JobPayload
4. Rust merges with hardcoded SKIP_DIRS / SKIP_EXTENSIONS
5. User entries are additive --- they cannot remove built-in entries
```

UI design rationale

The Settings page is accessible via a gear icon in the navbar — not as a third tab alongside Scanner and Header Audit. This follows the universal convention for settings in professional software (VS Code, GitHub, every SaaS tool). Settings is not a primary feature; it is configuration. Promoting it to a tab would imply equal weight with the core scanning workflow.

The three entry types are color-coded: purple for directory, teal for extension, amber for file. Grouped display makes the list scannable at a glance.

Industry Research: Vibe Coding vs Our Workflow

This section is based on a thorough review of 40+ sources published between December 2025 and March 2026, covering academic research, industry surveys, security audits, and practitioner accounts.

What most vibe coders get wrong

The most common failure pattern is starting without architectural planning — features are generated in isolation with no coherent structure. One developer described the result as "vibe-coded messes — codebases that look like they were written by multiple junior developers, all using completely different approaches." A March 2025 industry analysis found that AI-generated code often lacks structure, documentation, and clarity necessary for long-term maintenance.

Refactoring vibe-coded projects reveals unclear dependencies, inconsistent naming, and architecture held together by duct tape. The looseness that made vibe coding exciting becomes the same thing that makes scaling it unsafe.

Large-scale vibe coding fails when architecture is implied rather than enforced. Modular monoliths outperform pure microservices early on — boundaries matter, documentation matters, and without constraints, generation turns noisy.

What we did instead

> **Architecture before code.** The folder structure, Docker architecture, and IPC contract between Node and Rust were defined before writing a single line of implementation code.

> **Modular monolith.** The `src/modules/` structure with explicit route registration in `index.ts` means every module (auth, scanner, network, settings) has a clear boundary and a single entry point. No cross-module imports, no shared mutable state.

> **Single source of truth for types.** `types.ts` mirrors `types.rs` exactly. The IPC contract is a formal agreement between the two services, not an assumption.

> **Shared utilities, not duplicated logic.** `verifyToken` lives in one place and is borrowed by both the middleware and the SSE handler. Authentication logic was never copied.

The security gap

Analysis of real vibe-coded applications shows that 53 % of teams that shipped AI-generated code later discovered security issues that passed initial review. Research found that 86 % of AI-generated code fails XSS defense mechanisms, and 45 % introduces OWASP Top 10 vulnerabilities. Recurring patterns include: authentication logic silently altered during iterative prompting, authorization checks missing or inconsistently applied, and secrets propagated to client-side code.

Against this our approach was deliberately security-first from the start: HTTP-only cookies, two separate JWT secrets, path traversal rejection via Zod, read-only Docker bind mounts, and manual JWT verification in the SSE handler precisely because `EventSource` bypasses standard middleware. When the SSE endpoint rejected valid sessions, we fixed the root cause — we did not loosen the auth check.

What the research says separates good AI-assisted development

The most effective workflow, according to practitioners and researchers alike, is: architecture document first, then code. The developer's role becomes critical evaluation — asking questions such as "what if we simplified this component?" and "what happens if the data size became 10x?"

The skill to develop in 2026 is not writing better prompts. It is knowing how to judge generated code with the same attention you would give a junior developer's pull request — efficiently, but not blindly. Those who master this balance double their productivity. Those who ignore it build houses of cards.

Senior developers benefit most from AI-assisted development precisely because they can evaluate AI output effectively. They are not merely accepting output — they are critically evaluating it, refining it, and integrating it into robust systems. The senior developer's value has transformed from being a master coder to being a master architect and orchestrator.

The honest gap

We have the architecture, the modular boundaries, the security-first approach, and the documentation. What we do not have are automated tests (unit and integration) or static analysis tooling. This is the gap identified in the research as critical: "AI optimizes for local correctness, not systemic soundness. It gives you functions, not frameworks. Snippets, not structure. It can autocomplete your method, but it cannot reason about whether that method even belongs in this class, this module, or this service."

Automated tests are the verification layer that catches what static analysis cannot: broken auth flows, exposed data through legitimate endpoints, and logic errors that only appear at runtime. This is documented in the v0.3.0 roadmap.

Bottom line

What separates our workflow from typical vibe coding is that we treated the AI as a fast junior developer executing decisions we had already made architecturally — not as an autonomous system making those decisions for us. Andrej Karpathy, who coined the term, later hand-coded his own project entirely after finding that AI agents "just didn't work well enough." The developers succeeding with AI in 2026 are not the ones blindly accepting every suggestion.

They are the ones who have developed systematic approaches to AI-assisted development and maintained engineering judgment over every decision that matters.

v0.2.0 Feature Summary

Feature	Status
Health check indicator	Complete — gear + API gateway pattern
SSE progress streaming	Complete — real spinner, buffer parsing, scan thread
Filesystem watcher	Built, disabled — WSL2 inotify limit
Severity levels	Complete — Critical / High / Medium per rule
Shannon entropy scoring	Complete — capture group bug fixed
Extended patterns (14 rules)	Complete — AWS, GitHub, Stripe, Google, Slack, NPM, SendGrid, Twilio
Stopword filtering	Complete — global + per-rule
Configurable ignore list	Complete — SQLite, gear icon UI, three entry types
Terminal bracket logo	Complete — SVG, favicon, navbar lockup

Part 3 of 3. Notes accumulated during the v0.2.0 build session, 27 March 2026.